

Fiber Offers

Functional Requirements Document —
numbered, testable requirements with
acceptance criteria.

Document	Functional Requirements Document (FRD)
Category	Wallet & Payment UX Infrastructure
Author	Leo — Dilamme / Independent
Version	1.0
Status	Draft for Hackathon Submission

Table of Contents

- 1. 1. Purpose & How to Read This Document
- 2. 2. Offer Creation (FR-OC)
- 3. 3. Offer Resolution (FR-OR)
- 4. 4. Payment Execution (FR-PY)
- 5. 5. Recurrence (FR-RC)
- 5. 6. Fiber Address (FR-FA)
- 7. 7. SDK & UI Components (FR-SDK)
- 3. 8. Security & Trust (FR-SEC)
- 9. 9. Traceability Matrix (Requirements → Deliverables)

1. Purpose & How to Read This Document

This Functional Requirements Document (FRD) breaks the Fiber Offers product down into discrete, testable requirements. Each requirement has an ID, a priority (MoSCoW), and acceptance criteria that double as a QA checklist during the hackathon build. Read alongside the PRD (why we're building this) and the TRD (how the system is architected).

PRIORITY KEY

- MUST** required for a valid submission
- SHOULD** strongly increases score/usefulness
- COULD** nice-to-have if time allows
- WON'T** explicitly out of scope this cycle

ID SCHEME

- FR-OC-x** Offer Creation
- FR-OR-x** Offer Resolution
- FR-PY-x** Payment Execution
- FR-RC-x** Recurrence
- FR-FA-x** Fiber Address
- FR-SDK-x** SDK / UI Components
- FR-SEC-x** Security & Trust

2. Offer Creation (FR-OC)

FR-OC-1 — Create Offer from node identity **MUST**

The SDK shall provide a `createOffer()` function that fetches the caller's Fiber node public key via RPC and constructs a schema-v1-compliant Offer payload.

Acceptance criteria: Given a running Fiber node, calling `createOffer({ description, assets })` returns a signed, encoded Offer string within 3 seconds.

FR-OC-2 — Amount bounds **MUST**

Offer creation shall accept optional `amount_min` / `amount_max`. If both are omitted, the Offer is treated as payer-specified amount ("tip jar" mode).

Acceptance criteria: An Offer created with no amount fields resolves successfully when a payer supplies any positive amount; an Offer with bounds rejects out-of-range resolution requests with a clear error.

FR-OC-3 — Multi-asset declaration **SHOULD**

An Offer may declare more than one accepted `AssetSpec` (e.g. CKB or a specific UDT).

Acceptance criteria: A payer wallet supporting only one of the declared assets can still successfully resolve and pay using that asset.

FR-OC-4 — Signature integrity **MUST**

Every created Offer must be signed over its canonical serialization using the node's existing key material.

Acceptance criteria: Mutating any field after signing and attempting registration is rejected by the resolver with a signature-mismatch error.

FR-OC-5 — Offer registration **MUST**

Created Offers shall be registered with the resolver service via `POST /offers`, returning a stable `offer_id` and shareable encoded string + QR payload.

Acceptance criteria: Registration is idempotent for an identical payload (re-registering returns the same `offer_id`).

3. Offer Resolution (FR-OR)

FR-OR-1 — Fetch Offer metadata **MUST**

`GET /offers/:id` shall return the decoded, human-readable Offer metadata (description, assets, bounds, recurrence if present) without requiring authentication.

Acceptance criteria: Response matches the originally registered payload byte-for-byte on the signed fields.

FR-OR-2 — Mint invoice on demand **MUST**

`POST /offers/:id/invoice` shall call the recipient's Fiber node RPC to mint a fresh, single-use invoice matching the requested (and bound-validated) amount and asset.

Acceptance criteria: Two sequential calls to this endpoint for the same Offer return two distinct invoices, each independently payable.

FR-OR-3 — Reusability across sessions **MUST**

An Offer already resolved and paid once shall remain resolvable indefinitely (unless `single_use` is true or it has been explicitly revoked).

Acceptance criteria: The demo script pays the same published Offer twice from two independent wallet sessions without any regeneration step.

FR-OR-4 — Graceful failure on unreachable node **SHOULD**

If the recipient's node cannot be reached to mint an invoice, the resolver shall return a structured error (not a silent timeout) that the SDK surfaces to the payer clearly.

Acceptance criteria: Simulated node downtime produces a distinguishable "recipient temporarily unavailable, try again" state in the demo app UI.

FR-OR-5 — Offer revocation **SHOULD**

`DELETE /offers/:id` shall permanently invalidate an Offer given a valid signature from the original node key.

Acceptance criteria: Post-revocation resolution attempts return a clear "offer revoked" error rather than a generic failure.

4. Payment Execution (FR-PY)

FR-PY-1 — Unmodified settlement path **MUST**

Payment of a resolved invoice shall use Fiber's existing channel settlement mechanism with zero modification.

Acceptance criteria: A resolved invoice from this system is payable by any standard Fiber-compatible wallet with no Offer-awareness at all.

FR-PY-2 — Payment confirmation propagation **SHOULD**

Upon successful settlement, the resolver shall record the payment against the Offer's history for reconciliation and recurrence tracking.

Acceptance criteria: The Offer's recurrence-status endpoint reflects a newly completed payment within 5 seconds of settlement.

5. Recurrence (FR-RC)

FR-RC-1 — Recurrence declaration **SHOULD**

An Offer may include a `RecurrenceSpec` with interval, per-cycle amount, and either `cap_cycles` or `spending_cap_total` (at least one cap is mandatory).

Acceptance criteria: Registration of a recurring Offer without any cap field is rejected by the resolver.

FR-RC-2 — Automatic cycle triggering **SHOULD**

The scheduler shall trigger a resolution + payment attempt automatically when a recurrence cycle becomes due.

Acceptance criteria: In a simulated/accelerated-time demo, a monthly Offer triggers a second payment cycle automatically without manual intervention.

FR-RC-3 — Cap enforcement **MUST**

The scheduler shall refuse to trigger further cycles once `spending_cap_total` or `cap_cycles` is reached.

Acceptance criteria: Attempting to exceed the cap in a test scenario results in a blocked cycle and a user-facing notification, not a silent overspend.

FR-RC-4 — User revocation of standing approval **MUST**

The payer wallet shall provide a one-tap action to revoke a standing recurring approval at any time.

Acceptance criteria: Revoking mid-cycle prevents all subsequent scheduled attempts for that Offer on that payer's wallet.

6. Fiber Address (FR-FA)

FR-FA-1 — Well-known resolution endpoint **SHOULD**

`GET /.well-known/fiberoffer/:user` shall resolve a registered username to its canonical Offer.

Acceptance criteria: Entering `alice@domain` in the demo payer app successfully resolves to Alice's Offer and completes a payment.

FR-FA-2 — Username registration **SHOULD**

A recipient shall be able to bind a chosen username to a specific `offer_id` under a domain they control.

Acceptance criteria: Attempting to bind a username already claimed under the same domain is rejected.

7. SDK & UI Components (FR-SDK)

FR-SDK-1 — Drop-in QR/link component **MUST**

The SDK shall ship a React and React Native component that renders a given Offer as a scannable QR code and a copyable link, with no required styling configuration.

Acceptance criteria: A second, independent demo app can render a working Offer QR by importing the component and passing an `offer_id`, in under 30 minutes of integration effort.

FR-SDK-2 — Resolution + pay helper **MUST**

The SDK shall expose a single high-level function that, given a scanned Offer string, performs resolution and returns a ready-to-pay invoice object.

Acceptance criteria: Calling the helper end-to-end against the devnet resolver returns a valid, payable invoice.

FR-SDK-3 — Recurrence approval UI **SHOULD**

The SDK shall include a reusable approval screen component that clearly displays per-cycle amount, interval, and total spending cap before a user approves a recurring Offer.

Acceptance criteria: The approval screen cannot be dismissed with an approval action unless the cap is visibly rendered.

8. Security & Trust (FR-SEC)

FR-SEC-1 — Signature verification on registration **MUST**

The resolver shall verify the Offer's signature against the claimed `node_id` before accepting registration.

Acceptance criteria: A tampered or invalidly signed Offer is rejected with HTTP 400 and a descriptive error.

FR-SEC-2 — Authenticated minting RPC **MUST**

The resolver-to-node call that mints an invoice on the recipient's behalf shall be authenticated so only the legitimate Offer owner's registered service can trigger it.

Acceptance criteria: An unauthenticated or mismatched-credential attempt to mint on behalf of another node's Offer is rejected.

FR-SEC-3 — No custodial fund handling **MUST**

At no point shall the resolver service hold, forward, or have signing authority over user funds.

Acceptance criteria: Code review / architecture confirms all settlement occurs directly between payer and recipient Fiber nodes.

9. Traceability Matrix (Requirements → Deliverables)

Requirement group	PRD section	TRD section	Demo evidence
Offer Creation	\$6.1, \$8.1	\$3, \$4.1	Live creation of a merchant Offer
Offer Resolution	\$6.1, \$9.2	\$4.2, \$5	Same Offer paid twice
Recurrence	\$6.2, \$9.3	\$3.3, \$4.3	Simulated recurring cycle firing automatically
Fiber Address	\$6.3	\$5	Payment via <code>user@domain</code> instead of raw Offer string
SDK/UI	\$8.1	\$2	Second demo app integrating the QR component
Security	\$13	\$6	Tampered-signature rejection test shown live or in test suite

End of Functional Requirements Document. Together with the PRD and TRD, this package is intended to give a reader complete, end-to-end understanding of Fiber Offers — why it exists, how it's architected, and exactly what "done" means for the hackathon submission.